

Обмен данными между процессами

Manager, Proxy

Manager. Встроенные типы данных

- ▶ Semaphore
- ▶ BoundedSemaphore
- ▶ Condition
- ▶ Event
- ▶ Lock
- ▶ RLock
- ▶ Namespace
- ▶ Queue
- ▶ Array
- ▶ Value
- ▶ dict
- ▶ list

```
from multiprocessing import Manager, Process

def foo(x):
    print('Hello! ',x)

if __name__ == '__main__':
    M = Manager()
    L = M.list()
    L.append(1)

    L.append(1)

    p = Process(target = foo, args=(L,))
    p.start()
    p.join()
```

Объект пользовательского класса

```
from multiprocessing import Manager, Process

class MyAwesomeClass:
    def __init__(self):
        self.x = 100

    def __str__(self):
        return str(self.x)

def foo(obj):
    obj.x = 501
    print('Hello! ',obj)

if __name__ == '__main__':
    MAC = MyAwesomeClass()

    p = Process(target = foo, args=(MAC,))
    p.start()
    p.join()

    print('Here I am: ', MAC)
```

```
>> Hello! 501
>> Here I am: 100
```

Ой (((

Объект пользовательского класса

```
from multiprocessing import Manager, Process

class MyAwesomeClass:
    def __init__(self):
        self.x = 100

    def __str__(self):
        return str(self.x)

def foo(obj):
    obj[0].x += 100
    print('Hello! ',obj[0])

if __name__ == '__main__':
    M = Manager()
    L = M.list()

    MAC = MyAwesomeClass()
    L.append(MAC)

    p = Process(target = foo, args=(L,))
    p.start()
    p.join()

    print('Here I am: ', L[0])
```

```
>> Hello! 100
>> Here I am: 100
```

ОЙ (((

Объект пользовательского класса

```
from multiprocessing import Manager, Process
```

```
class MyAwesomeClass:  
    def __init__(self):  
        self.x = self.baz()  
  
    def baz(self):  
        for i in range(10):  
            yield i*i
```

```
def foo(obj):  
    print('Hello! ',obj[0])
```

```
if __name__ == '__main__':  
    M = Manager()  
    L = M.list()
```

```
    MAC = MyAwesomeClass()  
    L.append(MAC)
```

```
    p = Process(target = foo, args=(L,))  
    p.start()  
    p.join()
```

>> TypeError: cannot pickle
'generator' object

ОЙ-ОЙ (((

Объект пользовательского класса

```
from multiprocessing import Manager, Process
from multiprocessing.managers import BaseManager, NamespaceProxy

class MyAwesomeClass:
    def __init__(self):
        self.x = 100

def foo(obj):
    obj.x = 200
    print('Hello! ',obj.x)

class MyManager(BaseManager):
    pass

class TestProxy(NamespaceProxy):
    _exposed_ = ('__getattr__', '__setattr__', '__delattr__')

MyManager.register('MyAwesomeClass', MyAwesomeClass, TestProxy)

if __name__ == '__main__':
    M = MyManager()
    M.start()

    MAC = M.MyAwesomeClass()

    p = Process(target = foo, args=(MAC,))
    p.start()
    p.join()

    print('Here I am: ', MAC.x)
```

>> Hello! 200
>> Here I am: 200

УРА!

Создание Proxy и что это такое?

<https://docs.python.org/3/library/multiprocessing.html#programming-guidelines>

- ▶ Менеджер создает серверный процесс, в котором размещается централизованная версия объектов, которые могут совместно использоваться несколькими процессами.
- ▶ Объекты не являются «общими». Менеджер создает прокси-объект для каждого объекта, которым он управляет, и прокси-объекты совместно используются процессами.
- ▶ Прокси-объекты используются и работают точно так же, как исходные объекты, за исключением того, что они сериализуют данные, синхронизируют и координируют с централизованной версией объекта, размещенной в менеджере.

На что обратить внимание

- ▶ Функции внутри классов должны быть либо «exposed», либо обернуты в пользовательский proxy class;
- ▶ Нестандартные структуры данных не могут быть сериализованы по умолчанию;
- ▶ Вложенные структуры данных могут вызывать проблемы!
- ▶ Очень редко получаются «простые решения» с помощью стандартных функций;

Задача для тренировки

- ▶ Создать два класса Leader и Follower
- ▶ Объединить объекты этих классов в пары (3 пары)
- ▶ Объект Leader спит N секунд (задается в конструкторе класса), а затем передает в связанный с ним объект Follower случайное число $M < N$, затем снова засыпает.
- ▶ Объект Follower спит M секунд.
- ▶ Объекты проделывают 5 циклов.
- ▶ Объекты записывают все свои действия в файл в виде

Leader # 2

11:35.000 ушел спать на 4 секунды

11:39.001 проснулся

11:39.005 отправил Follower #2 число 3

11:39.007 ушел спать на 4 секунды